



Cheatsheet MIAE-215: Final Exam Study Guide on Pointers & Algorithms

Premium

Course: Mechanical, Industrial & Aerospace engineering (MIAE 215) 378 documents University: Concordia University Esen Lhagvasuren 2025/2026

Download

1 0 Save

This document covers essential programming concepts including pointers, file handling, mathematical patterns, data types, cor ...more

A. Pointers (HIGH PRIORITY)

Operator	Role
&	Address-of: Gets the memory address of a variable.
*	Dereference: Accesses the value at the pointer's address.

States: **NULL** (nullptr) is safe. **Wild** (uninitialized) is dangerous.
Arithmetic: `p + 1` moves by `sizeof(type)` bytes (e.g., +4 for `int*`).
Dynamic Memory: Must use `delete` for single items and `delete[]` for dynamic arrays.

B. File I/O (Requires <fstream>)

- ofstream:** Output (Writing). `outFile("data.txt");`
- ifstream:** Input (Reading). `inFile("data.txt");`
- Safety:** Always check if open: `if (outFile.is_open()) { ... }`

D. Mathematical Patterns (Engineering Applications)

Algorithms are often used to implement mathematical formulas in a loop.

Pattern	Logic	C++ Implementation
Accumulation	Adding a new value to a running total.	<code>total = total + value;</code>
Digit Generation	Shifting a number left and adding a digit.	<code>num = num * 10 + digit;</code>
Factorial	$N! = N * (N-1) * \dots * 1$	Recursion or loop - $O(N)$

```
#include <iostream>
#include <vector>
#include <algorithm> // Required for std::swap

using namespace std;
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    bool swapped; // Flag to check if any two elements were swapped in the inner loop
    // Outer loop runs (n-1) times. After i passes, the last i elements are in place.
    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        // Inner loop: Compares adjacent elements and performs swaps.
        // The comparison range shrinks on each iteration because the largest
        // element "bubbles up" to its final position at the end.
        for (int j = 0; j < n - 1 - i; j++) {
            // Compare the current element with the next element
            if (arr[j] > arr[j + 1]) {
                // Swap arr[j] and arr[j+1]
                swap(arr[j], arr[j + 1]); // Uses swap directly
                swapped = true;
            }
        }
        // Optimization: If no two elements were swapped by inner loop, then the array is
        // sorted.
        if (swapped == false) {
            break;
        }
    }
}
```

```
int main() {
    // 1. Initialize the array (using std::vector for modern C++)
    std::vector<int> numbers = {64, 34, 25, 12, 22, 11, 90};

    std::cout << "Original array: ";
    printArray(numbers);

    // 2. Call the sorting function
    bubbleSort(numbers);

    // [Image of bubble sort algorithm]

    // 3. Print the sorted array
    std::cout << "Sorted array: ";
    printArray(numbers);

    // Example 2: Already sorted array (shows O(n) best-case complexity due to
    // optimization)
    std::vector<int> sorted_numbers = {1, 2, 3, 4, 5};
    std::cout << "\nOriginal (sorted) array: ";
    printArray(sorted_numbers);
    bubbleSort(sorted_numbers);
    std::cout << "\nSorted array (fast exit): ";
    printArray(sorted_numbers);
}
```

Mock examSummaryQuiz

Ask a question about this document

```
// [image or bubble sort algorithm]

// 3. Print the sorted array
std::cout << "Sorted array: ";
printArray(numbers);

// Example 2: Already sorted array (shows O(n) best-case complexity due to optimization)
std::vector<int> sorted_numbers = {1, 2, 3, 4, 5};
std::cout << "\nOriginal (sorted) array: ";
printArray(sorted_numbers);
bubbleSort(sorted_numbers);
std::cout << "Sorted array (fast exit): ";
printArray(sorted_numbers);

return 0;
}
```

A. Basic Structure & Data Types

Type	Size (Typical)	Range	Notes
int	4 bytes	+/-2*10^9	Standard integer.
long long	8 bytes	+/-9*10^18	Use for large integer calculations.
double	8 bytes	Double precision	Preferred over float for most decimal math.
char	1 byte	Single character	Enclosed in single quotes (e.g., 'a').
bool	1 byte	true or false	Logical values.

Loop Type	When to Use	Syntax Reminder
for	Known number of iterations (e.g., array traversal).	for (init; condition; update) { ... }
while	Unknown iterations (e.g., reading a file, waiting for input). }	while (condition) { .. }
do-while	Must execute at least once (e.g., input validation).	do { ... } while (condition);

6. Arduino Programming & Mechatronics Structure

```
void setup() { /* Runs once */ }
void loop() { /* Runs repeatedly */ }
```

Key I/O Functions

- pinMode(pin, mode): Sets pin as INPUT or OUTPUT.
- digitalWrite(pin, val): HIGH or LOW.
- analogRead(pin): Reads value from 0 to 1023.
- delay(ms): Pauses program.

Standard structure:
#include <iostream>
using namespace std;
int main() { /* code */ return 0; }

B. Control Statements

Logic	Operator	Description
Equality	==	Is equal to.
Inequality	!=	Is not equal to.
AND	&&	Both conditions must be true.
OR		Either condition can be true.
NOT	!	Inverts the condition.

3. Arrays, C-Strings, and Functions

- A. Arrays (Static)**
- Indexing: Zero-based** (arr[0] is first; arr[size-1] is last).
 - Initialization:** int arr[] = {1, 2, 3};
- B. C-Strings (Requires <cstring>)**
- C-Strings are arrays of char terminated by a **null character** (\0).
 - char s[] = "Hi";** is stored as 'H','i','\0'

Function	Purpose
strlen(s)	Returns the length (excluding \0).
strcpy(dest, src)	Copies source string to destination. Check buffer size!
strcat(dest, src)	Appends source to destination.
strcmp(s1, s2)	Returns 0 if strings are identical.

C. Functions and Parameter Passing

Feature	Call by Value	Call by Reference (HIGH PRIORITY)
Syntax	void func(int x)	void func(int &x)
Effect	Changes a local copy only. Original variable is unaffected .	Changes the original variable in the caller.
Use	Read-only data; prevent side effects.	Swapping values; returning multiple results.

5. Arduino Programming & Mechatronics

Note: Assume you have access to these function signatures on the exam.

Structure

```
void setup() {
    // Runs once: Pin initialization
    pinMode(13, OUTPUT);
}

void loop() {
    // Runs repeatedly: Main logic/control
    digitalWrite(13, HIGH);
    delay(100);
}
```

Key I/O Functions

Function	Pin Mode	Value Range
pinMode(pin, mode)	Control	INPUT, OUTPUT
digitalWrite(pin, val)	Control	HIGH or LOW
digitalRead(pin)	Control	Returns HIGH or LOW

Mock exam

Summary

Quiz

Ask a question about this document



End of document

Was this document helpful?

1

0